

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>


typedef struct {
    char name[50];
    int priority; // 1 = Critical, 2 = Serious, 3 = General
} Patient;


// Define the Node structure for the Linked List
typedef struct Node {
    Patient patient;
    struct Node* next;
} Node;


// Function to create a new node
Node* createNode(char* name, int priority) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (!newNode) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    strcpy(newNode->patient.name, name);
    newNode->patient.priority = priority;
    newNode->next = NULL;
    return newNode;
}


// 1. Insert Operation - O(n) time complexity

```

```

void insertPatient(Node** head, char* name, int priority) {
    Node* newNode = createNode(name, priority);

    // If list is empty
    if (*head == NULL || (*head)->patient.priority > priority) {
        newNode->next = *head;
        *head = newNode;
    } else {
        Node* current = *head;
        // Traverse to find the correct position
        while (current->next != NULL && current->next->patient.priority <= priority) {
            current = current->next;
        }
        newNode->next = current->next;
        current->next = newNode;
    }
    printf("Patient '%s' (Priority %d) added to the queue.\n", name, priority);
}

// 2. Find Highest Priority Operation - O(1) time complexity
void findHighestPriority(Node* head) {
    if (head == NULL) {
        printf("The emergency queue is currently empty.\n");
        return;
    }
    printf("\n[Peek] Next Patient to Serve:\n");
    printf("Name: %s | Priority: %d\n\n", head->patient.name, head->patient.priority);
}

```

// 3. Delete Operation - $O(1)$ time complexity

```
void deleteHighestPriority(Node** head) {  
    if (*head == NULL) {  
        printf("The emergency queue is empty. No patients to serve.\n");  
        return;  
    }  
  
    Node* temp = *head;  
    printf("Serving Patient: %s (Priority %d)\n", temp->patient.name, temp->patient.priority);  
  
    *head = (*head)->next;  
    free(temp);  
}
```

// Function to display the entire queue

```
void displayQueue(Node* head) {  
    if (head == NULL) {  
        printf("Queue is empty.\n");  
        return;  
    }  
  
    Node* current = head;  
    printf("\n Current Emergency Queue \n");  
    int pos = 1;  
    while (current != NULL) {  
        printf("%d. %s (Priority: %d)\n", pos++, current->patient.name, current->patient.priority);  
        current = current->next;  
    }  
}
```

```
}
```

```
int main() {
```

```
    Node* emergencyQueue = NULL;
```

```
    insertPatient(&emergencyQueue, "John Doe", 3);    // General
```

```
    insertPatient(&emergencyQueue, "Alice Smith", 1); // Critical
```

```
    insertPatient(&emergencyQueue, "Bob Johnson", 2); // Serious
```

```
    insertPatient(&emergencyQueue, "Charlie Brown", 1); // Critical
```

```
    /*Display queue*/
```

```
    displayQueue(emergencyQueue);
```

```
    /* Find highest priority */
```

```
    findHighestPriority(emergencyQueue);
```

```
    /* Delete (serve) highest priority patient */
```

```
    deleteHighestPriority(&emergencyQueue);
```

```
    /* Display queue after serving */
```

```
    displayQueue(emergencyQueue);
```

```
    return 0;
```

```
}
```